# Code

```
dna = input("Enter DNA Sequence: ").upper()

# Length
print("\nSequence Length:", len(dna))

# GC Content g =
dna.count("G") c =
dna.count("C") gc = ((g + c) /
len(dna)) * 100
print("GC Content:", round(gc, 2), "%")

# Motif Search motif = input("Enter
Motif: ").upper() positions = []

for i in range(len(dna) - len(motif) + 1):
if dna[i:i + len(motif)] == motif:
        positions.append(i)

print("Motif Found At:", positions)

# Start Codon
start = dna.find("ATG")

if start != -1:
    print("Start Codon Found At:", start) else:
    print("Start Codon Not Found")

# Stop Codons
stop_codons = ["TAA", "TAG", "TGA"]

for stop in stop_codons:
pos = dna.find(stop)

    if pos != -1:         print(stop,
"Found At:", pos)     else:
        print(stop, "Not Found")

# Reverse Complement reverse
= dna[::-1] complement = ""

for base in reverse:
if base == "A":
```

```
        complement += "T"
elif base == "T":
complement += "A"     elif
base == "G":
complement += "C"     elif
base == "C":
complement += "G"

print("Reverse Complement:", complement)
```

# Output

```
Enter DNA Sequence: ATGGCGATCGATCGATCGTAGCGCGCTAA

Sequence Length: 29
GC Content: 55.17 %
Enter Motif: GCG
Motif Found At: [3, 20, 22]
Start Codon Found At: 0
TAA Found At: 26
TAG Found At: 18
TGA Not Found
Reverse Complement: TTAGCGCGCTACGATCGATCGATCGCCAT
```

Pratical No 2

## CODE

```
import pandas as pd

# Read Dataset
data = pd.read_csv("rna_seq.csv")

print("RNA-Seq Dataset\n")
print(data)

# Calculate Fold Change
data["Fold Change"] = data["Treated"] / data["Control"]

status = []

for fc in data["Fold Change"]:
if fc >= 2:
        status.append("Upregulated")
elif fc <= 0.5:
        status.append("Downregulated")
else:
        status.append("No Significant Change")

data["Status"] = status

print("\nDifferential Gene Expression\n")
print(data)
```

## OUTPUT

Control = 200
Treated = 100
Fold Change = 0.5
**Result:** Downregulated

Control = 50
Treated = 120
Fold Change = 2.4
**Result:** Upregulated

Control = 80
Treated = 60
Fold Change = 0.75
**Result:** No Significant Change

Pratical No 3

## CODE

```
seq= input("Enter the Protine Sequence:-").upper() print("\nProtein
Sequence:", seq)
amino ="ACDEFGHIKLMNPQRSTVWY"
count=[] for aa in amino:
    count=seq.count(aa)
if count>0:
print(aa,":",count) wt =
len(seq) * 110
print("total Weight is:",wt,"Dalton")
```

## OUTPUT

```
Enter the Protine Sequence:-
tswvtsacdghpqscadscadghpqsacdtswvtsacdghpqscadscadghpqsacdtswvtsacdghpqscadscadghp
qsacd

Protein Sequence:
TSWVTSACDGHPQSCADSCADGHPQSACDTSWVTSACDGHPQSCADSCADGHPQS
ACDTSWVTSACDGHPQSCADSCADGHPQSACD
A : 12
C : 12
D : 12
G : 6
H : 6
P  : 6
Q : 6
S : 15
T : 6
V : 3 W
: 3
total Weight is: 9570 Dalton
```

Pratical No 4

# Code

```
 import pandas as pd from sklearn.model_selection
import train_test_split from sklearn.ensemble import
RandomForestClassifier
from sklearn.metrics import accuracy_score

# Create Genomic Dataset data
= pd.DataFrame({
   "GeneFeature1": [2.5, 1.2, 3.1, 0.9, 2.8, 1.0, 3.2, 0.8, 2.9, 1.1],
   "GeneFeature2": [3.0, 0.8, 3.5, 1.1, 2.9, 0.7, 3.8, 1.0, 3.1, 0.9],
   "Class": [
      "Healthy", "Disease", "Healthy", "Disease", "Healthy",
      "Disease", "Healthy", "Disease", "Healthy", "Disease"
   ]
})

# Save Dataset as CSV
data.to_csv("genomic_data.csv", index=False)

# Read Dataset
data = pd.read_csv("genomic_data.csv")

print("Genomic Dataset\n")
print(data)

# Features
X = data[["GeneFeature1", "GeneFeature2"]]

# Target y =
data["Class"]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
   X,
   y,
test_size=0.3,
random_state=1,
stratify=y
)

# Create Model model =
RandomForestClassifier(
```

```
n_estimators=100,
random_state=1
)

# Train Model
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

print("\nActual Class")
print(y_test.values)

print("\nPredicted Class")
print(prediction)

# Accuracy
accuracy = accuracy_score(y_test, prediction)

print("\nAccuracy:", round(accuracy * 100, 2), "%")
```

## OUTPUT

Input features:
GeneFeature1 = 3.0
GeneFeature2 = 3.2 Expected
Class:
**Healthy**

Input features:
GeneFeature1 = 1.0
GeneFeature2 = 0.9 Expected
Class:
**Disease**